Name, Karan Singh

Roll, 251030611

Subject, Introduction to Reinforcement Learning (EE675), Assignment 1

Q1. (i) The principles behind each function

(a) Simulator Function

The code simulates one step of the environment by taking the current predator position, prey position, and an action, and returning the next positions along with a reward. The predator moves deterministically based on the chosen action using the move function, while the prey moves stochastically by randomly selecting one of the five possible actions with equal probability. The move function ensures valid movement within grid boundaries, if a move goes outside the grid, the agent stays in the same position. After both movements, a capture condition is checked, if the predator and prey occupy the same cell, a reward of 1 is assigned and the prey respawns randomly in any other cell except the predator's position. If no capture occurs, the reward is 0. This function essentially represents one step transition of the stochastic environment.

(b) Kernel function

The code constructs the transition kernel $P(s' \mid s, a)$by first enumerating all possible states as pairs of predator and prey positions and mapping each state to an index using state_to_index, which allows constant-time lookup when filling the matrix. For every state index s_idx and action index a_idx, a unique row in the kernel is computed using row_idx = s_idx * A + a_idx, effectively flattening the (state, action) pair into a single dimension. The predator moves deterministically using the move function, while the prey moves stochastically with equal probability $1/5$. For each possible prey move, the next state is computed, if capture occurs (same cell), the prey respawns uniformly over all other cells, so the probability $1/5$is further divided by the number of valid

respawn cells, ensuring total probability remains 1. If no capture occurs, the transition probability is directly added. The use of += is important because multiple prey actions can lead to the same next state, so probabilities accumulate. Overall, each row of the kernel becomes a valid probability distribution over next states.

Step 1, Initialize state and action space

- Generate all possible grid positions
- Form all states as (predator position, prey position)
- Create mapping, state → index

Step 2, Start main loops

- Loop over each state s
- Inside that, loop over each action a

Step 3, Compute row index

- Use, row_idx = s_idx * A + a_idx
- This converts (state, action) to unique row in kernel

Step 4, Move predator

- Apply action using move()
- Movement is deterministic

Step 5, Handle prey movement

- Consider all 5 prey actions

- Each has probability = $1/5$

Step 6, For each prey action

- Move prey using move()
- Compute next state

Step 7, Check capture condition

Case 1, Capture happens

- Predator and prey land on same cell
- Generate all grid cells
- Remove predator's cell
- Distribute probability uniformly,

$$\text{respawn prob} = \frac{1/5}{\text{number of cells}}$$

- Add probability to all valid next states

Case 2, No capture

- Only one next state
- Assign probability = $1/5$

Step 8, Update kernel

- Convert next state → column index
- Add probability using,
  kernel[row_idx, col_idx] += probability
- This ensures probabilities accumulate correctly

Step 9, Repeat

- Repeat for all prey actions
- Then all actions
- Then all states

Step 10, Final output

- Each row represents $P(s' \mid s, a)$
- Each row sums to 1 (valid probability distribution)
- Return kernel matrix

(c) Reward Function

The code constructs the reward function $R(s, a)$by first generating the complete state space as all possible pairs of predator and prey positions on the grid, and defining the action space for the predator. A reward matrix of size $S \times A$is initialized with zeros, where each entry corresponds to the immediate reward obtained when taking action a in state s. The movement of the predator is handled using the move function, which updates its position based on the chosen action while ensuring it stays within grid boundaries. For each state-action pair, the predator's next position is computed deterministically, and this is compared with the current prey position. If the predator's next position coincides with the prey's position, it indicates a capture, and the reward is set to 1; otherwise, the reward remains 0. Unlike the kernel function, this reward function does not consider prey movement or future states, it only depends on the immediate outcome of the predator's action. Thus, the matrix represents a deterministic mapping where each entry encodes whether a given action from a given state results in an immediate capture or not.

(d) Sample Policy Function

The code constructs a stochastic policy $\pi(a \mid s)$that balances exploitation and exploration by assigning higher probability to actions that move the predator closer to the prey while still allowing other valid actions. It first generates the full state space as all combinations of predator and prey positions and initializes a policy matrix of size $S \times A$, where each row represents a probability distribution over actions for a given state. The move function is used to compute the next predator position for each action while ensuring boundary constraints. For each state, only valid actions are considered (those that either change position or are 'stay'), and their resulting positions are stored. The Manhattan distance $\mid x_1 - x_2 \mid + \mid y_1 - y_2 \mid$is then computed between the next predator position and the prey position for each valid action, which serves as a measure of how close the predator gets to the prey. The minimum of these distances identifies the best actions (those that move the predator closest to the prey). These best actions collectively receive a total probability of 0.5, distributed equally among them, while the remaining valid actions share the remaining 0.5 probability equally. This ensures that the policy prefers moving toward the prey (exploitation) but still explores other actions, making it a stochastic and balanced policy.

(e) Induced Kernel Function

The code computes the policy-induced transition kernel $P^{\pi}(s' \mid s)$by combining the original transition kernel $P(s' \mid s, a)$with a given policy $\pi(a \mid s)$. The input kernel has shape $(S \times A, S)$, where each row corresponds to a specific (state, action) pair, and the policy has shape (S, A), representing the probability of taking each action in each state. The goal is to eliminate the action dimension by taking an expectation over actions. For each state s, the code iterates over all actions and retrieves the corresponding transition probabilities using row_idx = s_idx * A + a_idx, which maps the (state, action) pair to the correct row in the kernel. It then multiplies this transition vector

$P(s' \mid s, a)$by the probability of selecting that action $\pi(a \mid s)$, and accumulates the result into $P^{\pi}(s' \mid s)$. Mathematically, this implements the equation $P^{\pi}(s' \mid s) = \sum_a \pi(a \mid s) P(s' \mid s, a)$, which is a weighted sum of transition probabilities over all actions. The result is a reduced kernel of size (S, S), where each row represents the probability distribution over next states when actions are chosen according to the policy.

(f) Induced Reward Function

The code computes the policy-induced reward $R^{\pi}(s)$by taking the expected value of the immediate reward over all possible actions according to the given policy. The reward matrix has shape (S, A), where each entry represents the reward obtained for taking action $a$in state s, and the policy matrix (S, A)gives the probability $\pi(a \mid s)$of choosing each action in that state. For each state index s, the code iterates over all actions and multiplies the reward $R(s, a)$with the corresponding policy probability $\pi(a \mid s)$, accumulating the result into a single value $R^{\pi}(s)$. Mathematically, this implements the equation $R^{\pi}(s) = \sum_a \pi(a \mid s) R(s, a)$, which is simply the expected immediate reward under the policy. The output is stored as a column vector of shape $(S, 1)$, where each entry represents the average reward obtained when following the policy in that state.

(g) State Value Evaluation Function

The code computes the state-value function $V^{\pi}(s)$for a given policy by solving the Bellman expectation equation iteratively. It first reduces the action-dependent dynamics into state-only quantities by constructing the induced kernel $P^{\pi}(s' \mid s)$and induced reward $R^{\pi}(s)$. The induced kernel is computed by taking a weighted sum of transition probabilities across actions using row_idx = s * A + a, which maps each (state, action) pair to the correct row in the original kernel, and multiplying it with the policy probabilities $\pi(a \mid s)$. Similarly, the induced reward is computed as the expected reward $R^{\pi}(s) = \sum_a \pi(a \mid s) R(s, a)$, implemented efficiently using element-wise multiplication

and summation. Once these are obtained, the value function is initialized to zero and updated iteratively using the Bellman equation $V = R^{\pi} + \gamma P^{\pi} V$, where the matrix multiplication $P^{\pi}@V$computes the expected future value. The process continues until convergence, which is checked using the maximum absolute difference between successive value estimates falling below a small threshold ϵ. This ensures numerical stability and guarantees that the final Vapproximates the true value function under the given policy.

(h) Q-Value Evaluation Function

The code computes the action-value function $Q^{\pi}(s, a)$for a given policy by combining immediate rewards with expected future values. It first calls the state value evaluation function to obtain $V^{\pi}(s)$, which represents the expected long-term return from each state under the policy. Then, for each state-action pair, it computes the Q-value using the Bellman equation $Q(s, a) = R(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V(s')$. The indexing row_idx = s * A + a is used to extract the correct transition probability vector $P(s' \mid s, a)$from the kernel, which contains probabilities of transitioning to all possible next states. The expected future value is calculated using a dot product between this probability vector and the value function V, effectively computing $\sum_{s'} P(s' \mid s, a) V(s')$. This is then scaled by the discount factor $\gamma$and added to the immediate reward $R(s, a)$to obtain the final Q-value. The result is a matrix of size (S, A), where each entry represents the expected return of taking action $a$in state $s$and thereafter following the given policy.

(j) State Value and Runtime Analysis

[Since I have a very old and slow laptop I was not able to evaluate the required plots for larger values of N so I just plotted them for N = [5, 10] (for larger values my laptop kept crashing)]

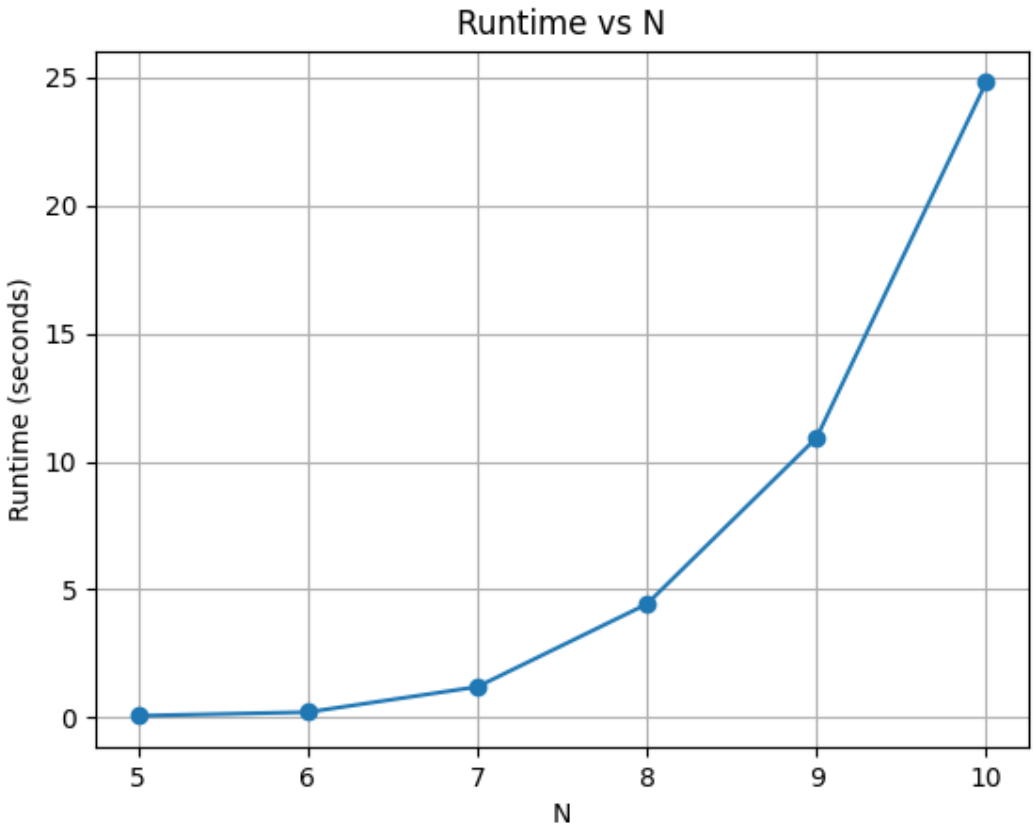

Figure: Runtime vs N

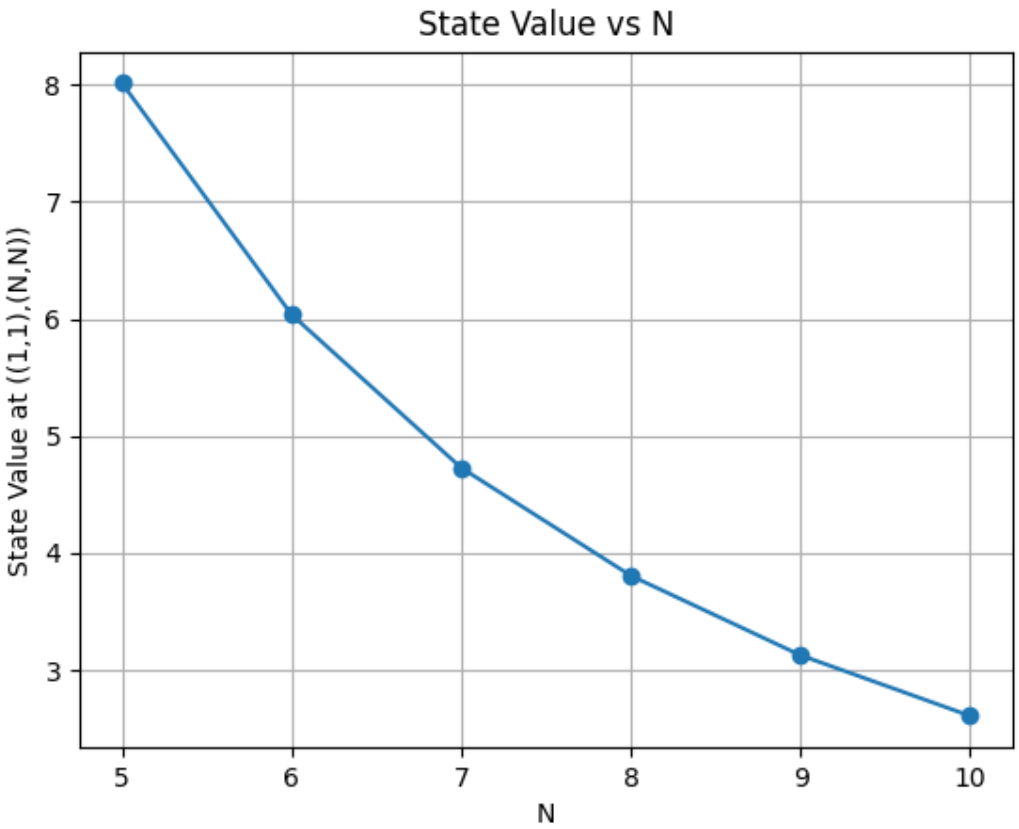

Figure: State Value vs N

The code evaluates how the state value and computational cost change with grid size $N$by repeatedly constructing the environment and solving for the value function. For each N, it generates the transition kernel $P(s' \mid s, a)$, reward function $R(s, a)$, and a sample policy $\pi(a \mid s)$, all of which depend on the full state space of size $S = N^4$(since both predator and prey can be anywhere on the grid). It then computes the value function V^πusing iterative Bellman updates. To extract a meaningful scalar, it selects a specific initial state $((1,1), (N, N))$, converts it to its index using a dictionary mapping (state → index), and records the corresponding value from V. At the same time, runtime is measured using timestamps before and after computation, capturing the total cost of building the model and solving it. This process is repeated for different values of N, and the results are plotted: one graph shows how the value of the chosen state changes with grid size, and the other shows how runtime increases. Since the number of states grows as $N^4$and the kernel size grows as $S \times A$, the runtime increases rapidly, which is why only small values of $N$are computationally feasible on limited hardware.